

Automating End-to-End Hybrid Query Processing: Benchmark, Solution, and Insights

Bo Li
Xiamen University
Xiamen, China
leebo@stu.xmu.edu.cn

Chenzhan Wang
Xiamen University
Xiamen, China
wangchenzhan@stu.xmu.edu.cn

Longkang Lin
Xiamen University
Xiamen, China
linlongkang@stu.xmu.edu.cn

Guoliang Li
Tsinghua University
Beijing, China
liguoliang@tsinghua.edu.cn

Chen Lin*
Xiamen University
Xiamen, China
chenlin@xmu.edu.cn

Abstract

Hybrid queries—natural language questions over structured data that require both database capabilities and LLM reasoning—have recently emerged as a prominent research topic. However, existing solutions remain overly dependent on manual workflows, and current benchmarks are limited in scale and diversity. To bridge this gap, we present (1) HyQBench, a large-scale benchmark with 60 ~ 90× more queries than prior work, built on 3× more databases; (2) AutoHyQ, an automated pipeline that can execute existing methods without manual intervention; (3) multi-dimensional, fine-grained evaluation metrics for comprehensive assessment. Through extensive experiments across multiple hybrid query approaches on diverse LLM backbones, we reveal their strengths and limitations, and identify research opportunities for advancing this emerging field. Our code and data are available at <https://github.com/XMUDM/HyQBench>.

CCS Concepts

• Information systems → Information systems applications.

Keywords

AI4DB, Hybrid Query, LLM, Database, Benchmark

ACM Reference Format:

Bo Li, Chenzhan Wang, Longkang Lin, Guoliang Li, and Chen Lin. 2026. Automating End-to-End Hybrid Query Processing: Benchmark, Solution, and Insights. In *Proceedings of the 32nd ACM SIGKDD Conference on Knowledge Discovery and Data Mining V.2 (KDD '26)*, August 09–13, 2026, Jeju Island, Republic of Korea. ACM, New York, NY, USA, 12 pages. <https://doi.org/10.1145/3770855.3817487>

1 Introduction

The rapid advancement of Large Language Models (LLMs) has significantly expanded the scope of database interactions by combining structured retrieval with semantic understanding, giving rise to **hybrid queries** as a promising direction. As illustrated in Figure 1,

*Chen Lin is the corresponding author.



This work is licensed under a Creative Commons Attribution 4.0 International License. KDD '26, Jeju Island, Republic of Korea
© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2259-2/2026/08
<https://doi.org/10.1145/3770855.3817487>

a hybrid query is a natural language question that declares information needs over relational databases and whose answer is either a value or a piece of text that simultaneously requires traditional structured data retrieval and the semantic processing capabilities of Large Language Models for unstructured and semi-structured data. This process is a more flexible and complex form of data retrieval that goes beyond the expressive power of standard SQL.

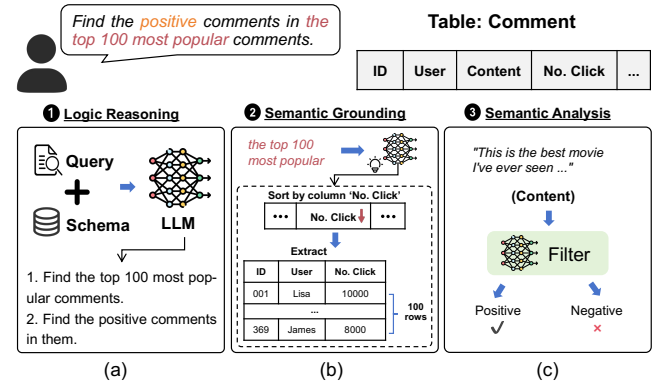


Figure 1: Illustration of a hybrid query: answering the query "Find the positive comments in the top 100 most popular comments" requires LLMs to (a) interpret the nested query logic and decompose the task, (b) infer implicit sorting criteria "No. Click" based on its common sense knowledge, and (c) perform sentiment analysis over textual content.

Existing hybrid query solutions fall into two paradigms: extending SQL syntax with LLM-based operators (e.g., DuckDB [35], BlendSQL [10], BigQuery [16]) or invoking sequences of independent LLM operators (e.g., LOTUS [31]). Effectiveness of these paradigms remains an open question because **neither achieves full end-to-end automation**, e.g., requiring hand-written scripts or manually selected and ordered operators. Consequently, evaluations are restricted to small-scale datasets such as TAG-Bench [1] and SWAN [45] (Table 1). Due to manual intervention, evaluations on these datasets clearly cannot fully capture the progress and bottlenecks of hybrid query solutions.

The goal of this work is to **provide a framework that unifies automated execution and large-scale benchmarking for**

hybrid query solutions. However, building large-scale benchmark datasets is inherently challenging: complex queries require significant computational resources to obtain correct answers. In addition, supporting different paradigms while ensuring correctness, scalability, and efficiency remains non-trivial.

To automatically construct a large-scale dataset with complex queries and corresponding answers, we propose a novel generation workflow: instead of first generating a query and then producing and verifying its answer, we begin by constructing a *reasoning process* (i.e., a sequence of operators and parameters), execute it to obtain the answer, and finally summarize a natural language query. We propose a set of strategies in this workflow to ensure that queries are logically consistent, executable, and yield non-empty answers. This design avoids the high cost of repeatedly executing and discarding invalid queries, allowing the scalable construction of large datasets.

The above generation workflow produces HyQBench, a benchmark dataset that is **larger, more diverse, and more challenging, and is equipped with fine-grained evaluation** to assess hybrid query solutions. As shown in Table 1, (1) HyQBench comprises 7,463 queries, making it 60 ~ 90× larger than existing benchmarks. (2) The queries span 15 databases across various domains (3 ~ 4× more than existing benchmarks), and combine functionalities from six general categories (1.5 ~ 3× more than existing benchmarks). (3) 41% of the queries are nested, making them more challenging to solve. (4) HyQBench provides multi-dimensional evaluation, including accuracy on intermediate reasoning steps.

Furthermore, we propose AutoHyQ, which supports **end-to-end autonomous implementation and evaluation** of both extended SQL-based and independent operator-based methods. Beyond automation, AutoHyQ also improves the performance of hybrid query solutions by enabling more accurate query execution plans through intent understanding and schema-aware reasoning. It further supports the extension to new operators via few-shot learning and enhances reliable execution through a reflection mechanism.

Through a thorough evaluation of various solutions across eight LLM backbones, we identify several notable insights.

(1) Existing solutions still fall far short of satisfactory performance: on HyQBench, the average accuracy reaches only about half of that reported on common text-to-SQL benchmarks. In particular, for hard queries, all evaluated methods exhibit a significant drop in answer accuracy of at least 38.06%.

(2) Independent operator-based methods outperform extended SQL-based methods, as the former executes queries step-by-step and reduces reliance on strict SQL syntax correctness, making them less error-prone.

(3) Both methods achieve higher accuracy on queries with semantic aggregation operators, since aggregation is fault-tolerant and local errors rarely affect overall trends; in contrast, performance is lowest on semantic joins, where LLM hallucinations often cause execution failures.

(4) There is also substantial room for improving the utilization of LLMs. ① Instruction tuning primarily improves format correctness, but has a limited effect on choosing the right operators or generating valid parameters. ② The cost of LLMs remains a concern—without optimization, each query can consume 2.59–6.03× the actual demand.

Table 1: Comparison of Different Benchmarks

	#Queries	#Functionalities	#Databases	Auto Construction	Auto Evaluation
TAG-Bench	80	4	5	✗	✗
SWAN	120	2	4	✗	✗
HyQBench	7463	6	15	✓	✓

2 Related Work

Table QA, Text-to-SQL, and hybrid queries constitute three distinct paradigms in database question answering. Table QA retrieves answers directly from tabular data, and Text-to-SQL translates natural language into SQL for structured querying. In contrast, hybrid queries combine databases with the semantic understanding and reasoning capabilities of LLMs to address richer information needs.

Table QA answers questions through direct operations on table content [14, 15, 30]. Models such as TAPAS [11] and Table-GPT [21] enhance LLMs’ capability to process tabular formats through pre-training and fine-tuning. To address the context-length bottleneck of large tables, retrieval-based methods have been developed, including hierarchical retrieval in ITR [22] and multi-level filtering in TableRAG [3]. Despite these advances, Table QA remains focused on locating content within tables and lacks support for multi-step logical reasoning such as joins, aggregations, and operator composition, which are fundamental to hybrid queries.

Text-to-SQL maps natural language queries to executable SQL [12, 19, 23]. Recent systems—including SQL-of-Thought [2], CHASE-SQL [32], etc [24, 34, 36, 44]—achieve strong performance with LLMs. Large-scale benchmarks such as WikiSQL [46], Spider [43], and BIRD [20] drive progress. However, SQL generation remains sensitive to syntax errors in complex scenarios [18], and the expressive power of standard SQL itself limits its ability to perform semantic interpretation or commonsense-driven data processing.

Hybrid queries leverage both structured database operations and the semantic reasoning capacity of LLMs. To support this paradigm, several systems integrate LLMs into database engines. BigQuery and DuckDB allow LLM invocation within SQL statements through custom interfaces. BINDER [6] exposes LLM functions to SQL/Python via a unified API. BlendSQL augments SQLite with LLM-enabled operators, extending standard SQL semantics. LOTUS [31] provides independent semantic operators via a DataFrame API rather than SQL. These systems improve interoperability between LLMs and structured data, forming the technical foundation for hybrid querying. Nevertheless, existing work often reuses datasets primarily designed to evaluate cross-source retrieval and integration, such as HybridQA [5] and OTT-QA [4]. They are not explicitly designed for hybrid query tasks and do not examine data processing based on operators.

3 HyQBench

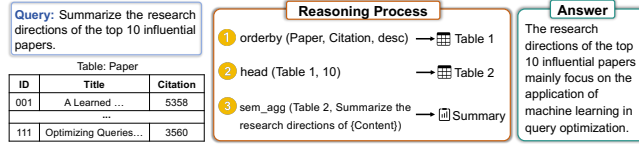
3.1 Dataset Overview

HyQBench consists of 7,463 instances, where each instance is a tuple $(Q, \mathcal{T}, P, A)$. As shown in Figure 2, Q is a natural language query, $\mathcal{T}$ is a set of target tables on which the query is executed, P is a valid and interpretable *reasoning process* which represents a correct way

Table 2: Operators and Corresponding Scenarios

Type	Standard	Semantic	Main Scenario
Filter	filter	sem_filter	Data Selection
Join	join	sem_join	Logical Connection
Sort	orderby	sem_topk	Semantic Sorting
Group	groupby	sem_cluster_by	Semantic Grouping
Aggregate	agg	sem_agg	Analysis / Summary
Map	map	sem_map	Information Extraction

to answer the query, and A is the corresponding answer obtained by executing P . The reasoning process P is provided as a fine-grained annotation to help analyze whether a hybrid query solution generates a correct reasoning path.

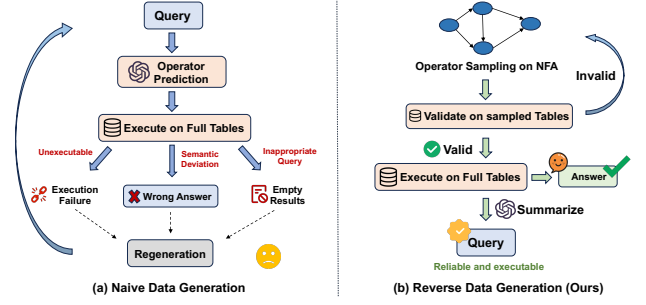
**Figure 2: An illustrative example in HyQBench**

Source tables. All target tables in $\mathcal{T}$ are drawn from a global table collection $\mathbb{T}$, which contains 79 tables from 15 databases. Most of them are from a widely used Text-to-SQL dataset BIRD [20], which is used to build prior hybrid-query benchmarks such as TAG-Bench [1] and SWAN [45]. Notably, HyQBench covers all 11 databases in the high-quality subset Mini-Dev released by BIRD, far exceeding prior benchmarks (4–5 databases), and significantly expanding the range of analysis scenarios represented in the benchmark. To address the lack of long texts in BIRD, we further incorporate four additional tables from scientific literature, news, and E-commerce [7, 25–27, 42]. These tables make HyQBench more representative of the data analysis demands involving LLMs in the real world. To ensure efficient evaluation, our dataset comprises sampled tables, where 1000 rows are drawn from each source table while preserving referential integrity.

Operators. For each instance, the reasoning process P is represented as a sequence of *operator* invocations, formally defined as $P = \langle o_1(\theta_1), o_2(\theta_2), \dots, o_k(\theta_k) \rangle$, where each operator o_i is a modular data-processing unit and θ_i denotes its parameters. Figure 2 illustrates an instance of operator invocation. The operators include standard SQL numerical operators and semantic operators that must be handled by LLMs. We define six general categories of operators commonly used across various analytic scenarios, as shown in Table 2. All operators are encapsulated in the same format as LOTUS [31].

3.2 Data Generation Framework

As shown in Figure 3(a), a naive pipeline for dataset construction typically follows the *query*→*answer* workflow: first generating a query, then letting the LLM autonomously plan an operation process, and finally executing that process to produce an answer. However, because complex queries often require multi-step execution and LLMs are prone to hallucinations, this pipeline can fail

**Figure 3: Comparison of Different Data Generation Methods**

in several ways. First, the process may include operator calls with formatting errors or disconnected steps, making it non-executable. Second, even when executable, the semantics may deviate from the intended query, yielding incorrect answers. Third, if the generated query itself is inappropriate, the process may produce empty results, which are unsuitable for benchmark datasets. In the naive pipeline, such issues are only discovered after lengthy database executions, forcing queries to be discarded and regenerated, thereby incurring *tremendous time overhead* in dataset construction.

In contrast, as shown in Figure 3(b), we propose a novel and automatic *reasoning process*→*answer*→*query* workflow. This order avoids searching through a vast space to find a plan that matches the predefined query, thereby eliminating wasted time from repeated executions of inappropriate queries. Moreover, we optimize the generation of reasoning processes by constructing and validating reasoning processes. We sample the steps of the process based on a Nondeterministic Finite Automaton (NFA) to ensure the logical integrity of the entire process. At every step, we sample an operator and immediately validate it on small samples from the database to ensure executability and that the process produces non-empty answers. If a newly appended step is invalid, it is discarded and resampled. Then, the reasoning process is executed to obtain an answer, from which the corresponding query is subsequently inferred. A key architectural advantage of our automated generation pipeline is its extensibility. To address domain-specific needs, researchers do not need to manually curate new datasets. By simply adding appropriate nodes to the NFA structure and supplying the corresponding operator descriptions in the prompts, our pipeline can automatically generate queries featuring these novel operators.

3.3 Reasoning Process Generation

The overall architecture is depicted in Figure 4. HyQBench covers both flat queries and nested queries. Since nested queries are essentially compositions of flat queries, we first construct reasoning processes for flat queries and then extend them to nested queries that involve multi-step and compositional reasoning.

Flat query. The standard execution order of a flat query’s operators is: *Join* → *Filter* → *Group* → *Aggregate* → *Map* → *Sort*. Therefore, we can obtain a flat query by traversing this order. We construct a Nondeterministic Finite Automaton (NFA) for operations as a directed graph, where each node allows three choices: the standard operator, the semantic operator, and “skip.” We perform

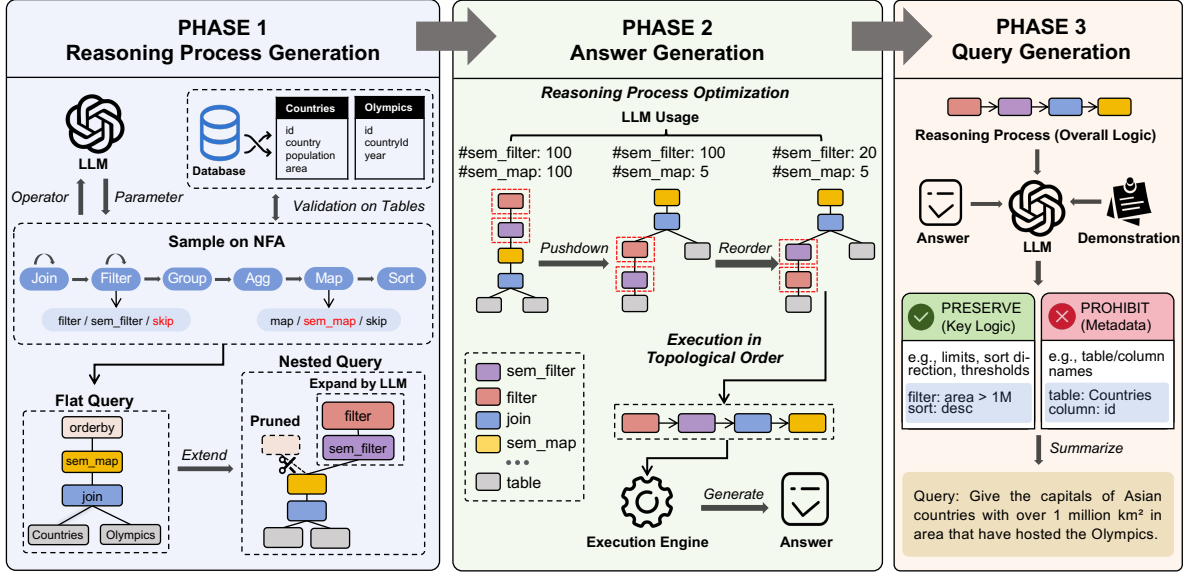


Figure 4: Overview of Data Generation

a depth-first search (DFS) on this graph to collect valid reasoning processes. To ensure coverage of semantic operators, we adopt an iterative generation scheme that requires the i -th semantic operator type to be selected in the i -th round of DFS. At each DFS step, we provide the selected operator, along with the table schemas, to DeepSeek-V3 [8] to generate concrete parameters. Operators are executed immediately to verify executability, and any operator that fails (e.g., due to execution errors or empty outputs) is discarded. We adopt the following strategies to further reduce validation cost: (1) we execute them on sampled tables of 100 rows. (2) WE de-duplicate target columns prior to execution, which is particularly effective for columns with low cardinality. (3) We reuse intermediate results: during DFS, we retain the outcome of each step in real time, so that when backtracking due to errors, we can resume from the stored results instead of re-executing from the beginning.

Nested query. To construct nested queries, we provide the reasoning process for a flat query, along with new related tables, to the LLM, which proposes multiple extension plans (up to four operators). The LLM also scores the plans for feasibility and coherence, and the best plan is executed to form a nested query.

The expanded plan may incorporate redundant operations or render existing operations redundant. For example, as shown in Figure 4, the LLM adds the operators `filter` and `sem_filter` to the flat query. This makes the original `orderby` step unnecessary because its output is not used by subsequent operators. We must ensure that the reasoning process is concise and includes only the necessary operations, so that the query inferred from it is correct and unambiguous. Thus, we adopt pruning on the expanded query plan. We treat the *final operator* (e.g., `sem_filter` in Figure 4) in the sequence as the final step, and initialize a preserved set with this operator. Then we traverse the sequence in reverse order and retain an operator if its output is used by any operator already preserved; otherwise, we prune it.

3.4 Answer Generation

After constructing the reasoning process, we execute the operators to produce answers. However, direct execution in the original order is extremely time-consuming and requires a large number of LLM tokens. Therefore, it is necessary to *optimize the reasoning processes* to reduce the amount of data processed by LLMs, thereby improving execution efficiency.

Inspired by classical database query optimization, we optimize the operator sequences. We first construct a query tree based on dependencies among operators, and then apply optimization strategies to adjust the positions of nodes in the tree. Finally, a topological sort is performed on the optimized query tree to produce the final execution sequences.

There is a significant difference in the cost between standard and semantic operators. For instance, a standard filter on 1,000 rows completes in milliseconds, whereas a `sem_filter` call may take approximately 5 seconds. Classical database optimizations, though highly effective in SQL engines, do not directly transfer to hybrid queries. For example, blindly pushing a `sem_filter` downward via predicate pushdown may inflate the number of LLM calls and increase cost. Thus, hybrid query optimization must explicitly consider the interaction between standard and semantic operators, aiming to minimize the input size of the latter while ignoring the cost of the former.

Reasoning Process Optimization. We focus on the most effective parts for reducing cost, namely optimizing the dominant filter and join operators, through two strategies: predicate push-down and operator re-ordering (details of the algorithms provided in Appendix B). (1) Predicate push-down. Extending classical rules, we present a refined predicate pushdown that makes operator-specific decisions rather than uniformly pushing down all predicates. As shown in Figure 4, `filter` and `sem_filter`, though logically equivalent,

are handled differently due to their sharply contrasting costs. (2) Operator re-order. We further optimize operator order among adjacent steps. Standard operators are always executed before their semantic counterparts. When two semantic operators appear consecutively, we order them by estimated cost, approximated by the number of distinct values in their target columns. Predicate pushdown and operator reordering are grounded in the fundamental equivalence rules of relational algebra, which inherently guarantee that the output remains logically identical. Our specific contribution lies in adapting the trigger conditions of these classic algorithms to account for the unique characteristics and high computational costs of semantic operators. This adaptation dictates when a swap or push-down is beneficial, optimizing execution efficiency while strictly preserving the proven logical correctness of the query plan. Section 5.5 presents evidence of the effectiveness of our optimization strategy.

3.5 Query Generation

Finally, we use DeepSeek-V3 to generate a corresponding natural language query for each reasoning process.

To ensure semantic consistency and query validity, we impose strict constraints on the generation process. We require that natural language queries generated be precisely aligned with the operators used in the reasoning process and not introduce ambiguity or inconsistent logic. We explicitly prohibit the mention of table names, column names, or other structural metadata in the generated query—these are expected to be inferred by the evaluated model. At the same time, key constraints, such as filtering thresholds, row limits, and sorting directions, must be preserved to ensure that the query remains executable and answerable. To help the LLM better adhere to these generation constraints, we include two high-quality demonstration examples in the prompt to guide it toward the desired generation format and semantic fidelity.¹

3.6 Validation

To ensure the benchmark’s reliability and consistency, we incorporate data validation across both the answer and query generation stages. During answer generation, we apply a double-execution validation mechanism to reduce errors from LLM randomness and hallucinations. Each semantic operator is executed twice independently, and samples with inconsistent results are discarded, as such discrepancies indicate ambiguity or instability in semantic processing. For query generation, we require LLM to reflect on each candidate, evaluating whether the query exhibits reasonable and natural phrasing, and can be reasonably answered through the model’s reasoning ability and enriched knowledge. This step excludes overly contrived, illogical, or unanswerable queries.

To further assess the dataset’s overall quality, we conduct a manual validation phase. Given the large scale of HyQBench, exhaustive human validation is practically infeasible, a constraint shared by most large-scale, established benchmarks in the community. Therefore, we randomly sample 200 query instances and present each query, along with its corresponding tables, to an experienced database expert, who annotates the operators required to solve it. We then compare the expert-annotated operator sequence with our

¹All prompts used in this paper can be found on [link]

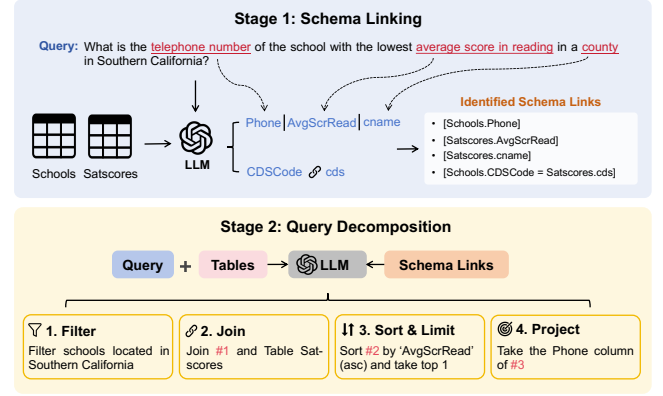


Figure 5: An Example of a Parser

reasoning process. The results show a high level of consistency: 93.5% of our operator plans fully match the expert’s judgment.

4 AutoHyQ

We aim to build an evaluation platform to assess the ability of different methods on hybrid queries. However, existing approaches rely on manual workflows and thus lack both a unified interface and an automated end-to-end evaluation paradigm. We introduce AutoHyQ, which allows different methods to be seamlessly integrated and automatically executed for fair and systematic comparison. We will also show in the experiments that this pipeline can increase the performance of hybrid query solutions. Based on the requirements of hybrid queries: accurate intent understanding, schema-aware reasoning, and reliable execution, AutoHyQ consists of four components: a **Preprocessor**, a **Parser**, a **Planner**, and an **Executor**.

Preprocessor. This component performs intelligent column selection to improve both effectiveness and efficiency. Real-world tables often contain a large number of attributes, many of which are irrelevant to the given query. Passing the full schema to an LLM will distract semantic understanding. We design a few-shot prompt that incorporates the query, database schema, operator descriptions, and annotated examples to guide LLMs in retaining only relevant columns. To ensure referential integrity, all primary and foreign keys are always preserved.

Parser. The Parser transforms a natural language query into a structured representation that captures its underlying intent, sub-tasks, and their dependencies. It consists of two subcomponents: (1) **Schema Linking.** Inspired by Text-to-SQL systems, we use schema linking to help models identify which table columns are implicitly referenced by the query. We adapt the prompt of DIN-SQL [33] to a hybrid query setting, enabling the model to locate relevant columns supporting semantic reasoning. (2) **Query Decomposition.** Based on the identified schema links, we prompt the LLM to decompose the query into a sequence of atomic reasoning steps following the Question Decomposition Meaning Representation (QDMR) formalism [41]. Each step corresponds to a single conceptual operation, and dependencies between steps are explicitly represented.

Planner. The Planner translates the parsed steps into executable operations. For extended SQL-based methods, we provide the LLM

with usage examples and prompt it to generate SQL statements directly. For independent operator-based methods, planning is performed incrementally: at each step, the model is presented with a set of available operators and prompted to select the appropriate one and configure its parameters. The system continuously tracks intermediate tables to support re-planning in the event of failure.

Executor. This component is responsible for interacting with the underlying database and executing SQL statements or operator invocations. To address possible errors during execution, we adopt an LLM reflection mechanism. We design specialized error prompts for common failure types and let the model iteratively refine its output. This enables the system to recover from many execution errors autonomously, significantly improving robustness.

5 Experiments

We conduct a comprehensive evaluation designed to answer the following research questions:

RQ1: What end-to-end performance can existing solutions achieve in hybrid query tasks when paired with different LLM backbones?

RQ2: How do existing solutions perform in intermediate steps of query processing, such as operator invocation and SQL execution?

RQ3: How do different components of AutoHyQ affect the results?

RQ4: What optimization directions can existing solutions adopt to reduce the cost of LLM invocations in hybrid query tasks?

Baselines. We conduct experiments on four baseline frameworks, each evaluated with at least six LLM backbones. First, we include LOTUS [31] to represent the *independent operator-based* paradigm. Since the original LOTUS implementation relies on manually written operator calls, we execute it through our AutoHyQ pipeline to enable large-scale evaluation. Second, we adopt BlendSQL [10] and iPDB [17] to represent the *extended SQL-based* paradigm. Like LOTUS, they were originally designed with manually specified SQL operations, and we also run them via AutoHyQ for consistency in evaluation. Finally, we consider BlendSQL combined with SQL-of-Thought [2], an advanced, *training-free* multi-agent framework that improves SQL generation. We treat this as a strong baseline from existing agentic approaches, which enables a direct comparison between our AutoHyQ-based design and the best available agentic alternative. We select six representative LLM backbones: GPT-5.1 [29], GPT-4.1 [28], DeepSeek-V3 [8], Qwen-Max [38], Gemini-2.5-Flash [9], and the lightweight open-source Qwen3-8B [39]. To examine whether lightweight models can benefit from task-specific adaptation, we include two fine-tuned variants, Qwen3-8B-sft and Qwen3-4B-sft, which are applied in the LOTUS+AutoHyQ setting, since our generated data can be directly leveraged for fine-tuning.

Implementation Details. (1) To ensure fairness, we provided explanations and demonstrations of the new operators in the extended SQL syntax for LLMs in BlendSQL+SQL-of-Thought. (2) All methods use GPT-4o-mini [28] as the underlying model for data processing. To ensure accuracy, we do not use proxy models in LOTUS. (3) The LLM fine-tuning comprises three epochs using approximately 5000 samples. We adopt the parameter-efficient fine-tuning method LoRA [13], with a LoRA rank of 8, a learning rate of $1e-4$, and a batch size of 16.

5.1 Metrics

We use five metrics that measure different aspects of performance, including end-to-end correctness, reasoning quality, operator invocation reliability, and execution efficiency.

Answer Accuracy (AC). Answer accuracy is the most direct end-to-end metric for evaluating the overall performance of hybrid query methods. Our dataset includes two answer types: (1) *Structured Answers*: tabular results generated through filtering, sorting, or transformation operations; and (2) *Unstructured Answers*: numeric values or free-form text, typically resulting from aggregation (e.g., textual summaries generated by the `sem_agg` operator). For structured answers, because solving hybrid queries may involve thousands of LLM calls and LLM outputs are inherently non-deterministic, an exact match is not an appropriate standard. We adopt *precision* and *recall* as the evaluation basis², and a structured answer is deemed correct if and only if both precision and recall exceed 0.9. For unstructured answers, due to the different representations of numerical results, a numerical result is considered correct if it either exactly matches the ground truth or is judged by GPT-4.1 to be numerically consistent with it. Similarly, the correctness of free-form text is assessed based on its semantic consistency with the reference answer, as determined by GPT-4.1.

Operator Precision (OP). This metric is defined as the ratio of correctly predicted operators (those appearing in both the model output and the ground truth) to the total number of operators predicted by the model. Formally:

$$OP = \frac{1}{N} \sum_{i=1}^N \frac{|\text{Operator}(P_i) \cap \text{Operator}(G_i)|}{|\text{Operator}(P_i)|} \quad (1)$$

Here, $\text{Operator}(P_i)$ denotes the set of operators called by the model for the i -th query, and $\text{Operator}(G_i)$ denotes the set of operators in the ground truth reasoning process for the same query. Importantly, we treat repeated operators with strict cardinality matching. For example, if the ground truth calls the filter operator only once, even if the model executes it multiple times, it is counted only once, ensuring that over-predictions are penalized. The variable N denotes the total number of test samples.

Operator Recall (OR). This metric is defined as the ratio of correctly predicted operators to the total number of operators in the ground truth. Formally:

$$OR = \frac{1}{N} \sum_{i=1}^N \frac{|\text{Operator}(P_i) \cap \text{Operator}(G_i)|}{|\text{Operator}(G_i)|} \quad (2)$$

This metric assesses the model's ability to capture the operator requirements of a given query fully.

Operator Format Accuracy (OFA). OFA focuses solely on the correctness of the input format during operator calling—e.g., whether the number of parameters is correct, whether field names are valid, and whether semantic descriptions are complete. Formally:

$$OFA = \frac{\sum_{i=1}^N \sum_{j=1}^{n_i} F(o_{i,j})}{\sum_{i=1}^N n_i} \quad (3)$$

Here, $o_{i,j}$ denotes the j -th operator call in query i , and n_i is the total number of operator calls in that query. The function $F(o_{i,j}) \in$

²Precision and recall are calculated based on row matching

Table 3: Answer Accuracy (%) of Different Methods

Method	Model	Easy	Med.	Hard	Total
LOTUS (AutoHyQ)	GPT-5.1	58.89	36.25	26.59	44.62
	GPT-4.1	61.23	32.13	22.79	43.27
	Qwen-Max	58.10	31.55	17.73	40.96
	DeepSeek-V3	57.92	30.00	19.86	40.52
	Gemini-2.5-Flash	54.24	27.49	14.81	37.14
	Qwen3-8B	49.87	22.73	10.77	32.66
	Qwen3-8B-sft	55.80	25.98	14.74	37.15
	Qwen3-4B-sft	52.81	24.87	12.94	35.14
BlendSQL (AutoHyQ)	GPT-5.1	43.82	21.21	11.27	29.48
	GPT-4.1	39.86	17.72	8.45	25.98
	Qwen-Max	38.23	16.32	5.63	24.18
	DeepSeek-V3	38.23	18.18	6.34	25.08
	Gemini-2.5-Flash	40.09	21.91	7.75	27.68
	Qwen3-8B	32.40	12.12	2.11	19.38
iPDB (AutoHyQ)	GPT-5.1	37.88	15.15	7.46	24.89
	GPT-4.1	34.96	14.00	5.28	22.12
	Qwen-Max	33.23	13.98	4.93	20.73
	DeepSeek-V3	32.05	14.58	4.54	20.48
	Gemini-2.5-Flash	27.98	13.40	3.53	18.23
	Qwen3-8B	23.90	11.07	1.78	15.22
BlendSQL (SQL-of-Thought)	GPT-5.1	35.28	15.29	7.05	22.68
	GPT-4.1	36.44	16.45	10.55	24.17
	Qwen-Max	31.75	14.13	10.78	21.20
	DeepSeek-V3	37.06	18.18	6.34	24.58
	Gemini-2.5-Flash	30.59	12.97	7.20	19.70
	Qwen3-8B	26.55	5.80	0.00	13.86

{0, 1} indicates whether the operator call has a valid format. We compute this metric across the entire test set by aggregating counts rather than averaging across individual samples. This approach avoids weight imbalance caused by variations in the total number of operators among samples.

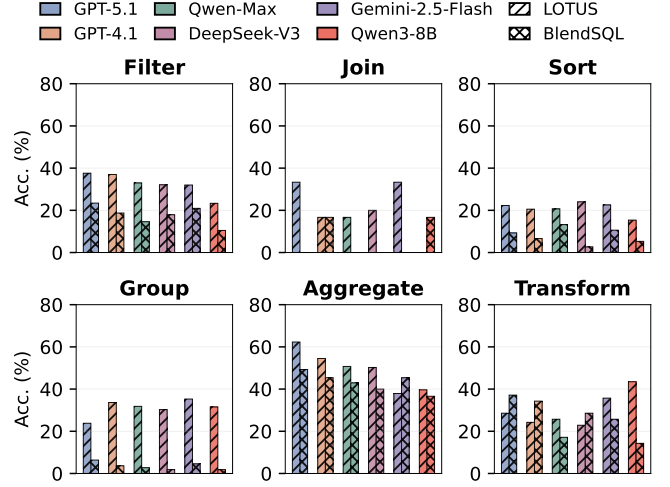
Cost Ratio (CR). This metric is defined as the ratio of token consumption between two solutions. Because of significant differences in data length, data volume, and processing steps across queries, using a relative ratio enables fair comparison across diverse scenarios. Formally:

$$CR = \frac{1}{N} \sum_{i=1}^N \frac{Cost(S1_i)}{Cost(S2_i)} \quad (4)$$

Here, $Cost(S1_i)$ and $Cost(S2_i)$ denote the total number of tokens consumed by the two solutions being compared for the i -th query.

5.2 End-to-End Performance

We first report the end-to-end answer accuracy in Table 3. We can see that the *current performance of existing methods for hybrid queries remains unsatisfactory*. Even the best-performing configuration achieves only **44.62%** total accuracy. This is substantially lower than the performance of state-of-the-art systems in standard SQL queries, such as Text-to-SQL (which typically exceed 80% [37, 40]), highlighting the intrinsic difficulty of hybrid query solving. *Using LOTUS as the underlying solution is significantly superior to using BlendSQL or iPDB under the same LLM backbone*. This advantage mainly stems from two factors. First, step-by-step operator invocation decomposes a hybrid query into a sequence of *local decisions*, reducing task difficulty. Second, operator invocation is easier than SQL generation in maintaining syntactic correctness (more details regarding executions will be discussed in Section 5.3). Furthermore,

**Figure 6: Accuracy of Different Query Categories**

on BlendSQL, AutoHyQ *consistently outperforms SQL-of-Thought* (more details regarding AutoHyQ ablation will be discussed in Section 5.4). This is mainly because SQL-of-Thought is designed around standard SQL syntax: its schema understanding and error-reflection mechanisms are tightly coupled with conventional SQL operators and execution rules, leaving it unable to adapt effectively to newly introduced operators.

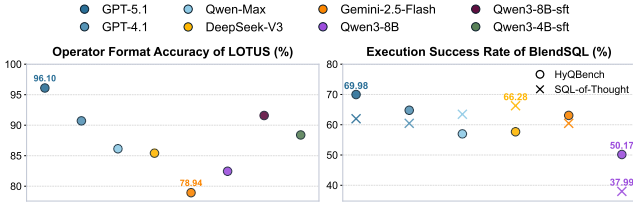
We group queries into three categories: Easy (≤ 3 operators), Medium (4–6 operators), and Hard (> 6 operators). In simple queries, the performance of LLMs is comparable to, and in some cases even exceeds, results obtained from manually written code or SQL statements in other benchmarks (e.g., TAG-Bench[1] reports an overall accuracy of about 55%, while in SWAN[45], HQDL achieves 40% and HQ-UDFs achieves 20.8%), demonstrating the effectiveness of AutoHyQ in enabling LLMs to solve hybrid queries automatically. However, accuracy drops sharply as query complexity increases, reflecting two key challenges: (1) harder queries involve multiple intents and broader operator coverage, and (2) data processing at each step may result in errors (as analyzed in Section 5.3), which may accumulate in longer reasoning chains. These observations also highlight the importance of our proposed benchmark HyQBench: it enables a comprehensive evaluation of how current LLM-based solutions perform on hybrid queries.

Our further error analysis shows that current LLMs can produce inaccurate answers due to three types of errors. (1) Operator misuse or faulty SQL generation: LLMs may select wrong operators, apply them incorrectly, or generate invalid SQL queries, leading directly to incorrect execution results. We will discuss this type of error further in Section 5.3. (2) Stochastic output: Due to the inherent instability of LLMs, hybrid queries—which involve multiple steps and large-scale data processing—are highly sensitive to errors. Even if a single step or instance produces an incorrect output, the entire final answer can be compromised. (3) LLM hallucination: LLMs may output factually incorrect judgments, even for simple queries.

As shown in Figure 6, the three types of LLM errors mentioned above lead to varying accuracy when answering queries involving different categories of operators. It should be noted that for

Table 4: Operator Precision and Recall of Different LLMs

Model	Operator Precision (%)				Operator Recall (%)			
	Easy	Med.	Hard	Total	Easy	Med.	Hard	Total
GPT-5.1	66.01	74.81	81.77	72.03	83.61	78.85	69.81	79.60
GPT-4.1	63.94	74.32	77.60	70.34	77.28	74.70	58.42	73.48
Qwen-Max	54.32	72.51	75.67	65.16	75.03	76.76	69.05	74.92
DeepSeek-V3	53.81	72.74	73.30	64.71	74.27	74.70	63.99	72.99
Gemini-2.5-Flash	48.40	62.67	63.31	56.64	76.46	73.20	61.90	72.98
Qwen3-8B	58.24	69.69	80.66	66.35	69.18	68.93	61.81	68.02
Qwen3-8B-sft	56.10	72.31	67.03	64.61	67.84	63.55	43.66	62.55
Qwen3-4B-sft	58.71	78.71	79.29	70.22	75.03	71.43	52.33	70.24

**Figure 7: OFA and SR of Different LLMs**

the results of BlendSQL, we only count the data that generated valid SQL statements. Due to the lack of corresponding syntax support for semantic-based sorting and grouping in BlendSQL, it performs poorly in these two areas. The results indicate that all models perform substantially better on summarization/analytical queries driven by *sem_agg* and on grouping queries driven by *sem_cluster_by* than on other query types. The explanation is intuitive: *sem_agg* is inherently tolerant to local processing errors because aggregate outputs depend on overall trends rather than on individual records, and *sem_cluster_by* obtains embeddings from a fixed text-encoding model and clusters semantics by similarity. By contrast, queries that require row-level precise processing are much more susceptible to stochasticity and hallucination—errors at individual steps propagate along the reasoning chain and are amplified, degrading final accuracy.

In short, our analysis shows that the primary bottleneck for current performance is the LLM’s reliability in fine-grained data processing. Improving output consistency and accuracy should therefore be a priority for future work on hybrid query systems.

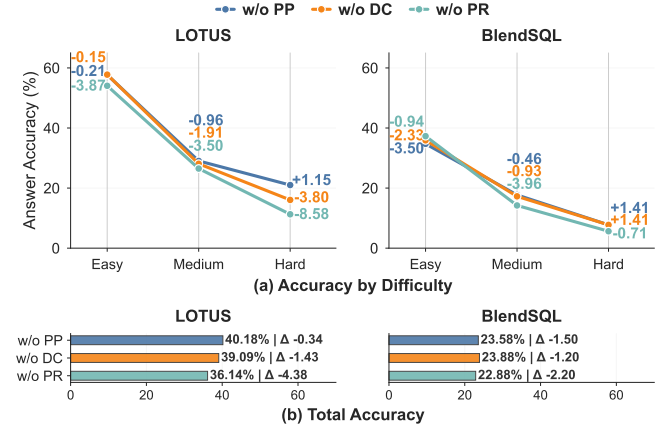
5.3 Intermediate Performance

Since the LOTUS-based method uses step-wise operator selection, and HyQBench provides fine-grained reasoning process annotations, we further analyze the operator-level behavior of LOTUS-based execution. Table 4 reports operator precision (OP) and operator recall (OR) for different LLMs. As query difficulty increases, most backbones exhibit *higher precision but lower recall* in operator selection. This reflects a conservative strategy: under uncertainty, LLMs tend to select only the most confident operators, but at the cost of omitting necessary steps. Fine-tuning significantly improves Qwen3 variants in answer accuracy (Table 3), yet its gains in operator selection remain less consistent. This suggests that SFT may primarily strengthen instruction following and parameter formatting rather than fundamentally improving operator-level reasoning.

We examine *syntax correctness*. Figure 7 reports operator format accuracy (OFA) for LOTUS and execution success rate (SR) for BlendSQL. OFA is consistently high, indicating that LLMs are generally reliable in producing syntactically valid operator calls. Moreover, fine-tuning improves OFA noticeably, indicating that the variants can better generate parameters in the correct format. Therefore, the dominant error sources of this method lie in operator selection and the content of parameters rather than formatting. In contrast, BlendSQL exhibits much lower SR, and its trend closely follows AC in Table 3, implying that the main bottleneck of extended SQL approaches is generating executable SQL programs.

5.4 Ablation Study on AutoHyQ

To assess the contribution of different components of AutoHyQ, we conduct an ablation study on the DeepSeek-V3. We remove three key components independently: the **Preprocessor (PP)**, the **Decomposition (DC)** step inside the **Parser**, and the entire **Parser (PR)**. The final results are summarized in Figure 8.

**Figure 8: Ablation Experiment of DeepSeek-V3**

Overall Effectiveness. Across all settings, ablating any component leads to an apparent decline in answer accuracy. The impact is substantially more pronounced for LOTUS than for BlendSQL. This is expected, as the latter is fundamentally bottlenecked by SQL execution success rate, while the former more fully benefits from improvements in reasoning structure, showing stronger stability and clearer degradation patterns when components are removed.

Impact of Different Components. The contributions of the three components are not uniform. Removing the Preprocessor yields the smallest drop in accuracy, suggesting that schema constraints from downstream planning still implicitly guide the model toward relevant fields. In contrast, removing the entire Parser causes the sharpest degradation, highlighting the central role of structural parsing in generating executable and logically consistent reasoning plans. The effects of removing only the decomposition step lie in between, demonstrating that fine-grained sub-goal decomposition is beneficial but less critical than schema-aware structural parsing.

Sensitivity to Query Difficulty. The influence of each component becomes more significant as query complexity increases. For LOTUS, removing the Query Decomposition component leads to

Table 5: Cost Ratio Relative to the Optimized Sequence

Model	Cost Ratio ↓		
	Prompt	Completion	Total
GPT-5.1	2.88	7.58	3.00
GPT-4.1	3.36	5.85	3.42
Qwen-Max	4.63	15.63	4.92
DeepSeek-V3	3.78	14.41	4.06
Gemini-2.5-Flash	4.21	7.56	4.30
Qwen3-8B	5.84	13.02	6.03
Qwen3-8B-sft	2.50	5.76	2.59
Qwen3-4B-sft	3.37	7.04	3.46

a **3.80%** drop in AC on hard queries, whereas removing the entire Parser results in an **8.58%** decrease. These results confirm that deep schema comprehension and step-wise reasoning are especially essential for resolving multi-intent and deeply nested hybrid queries.

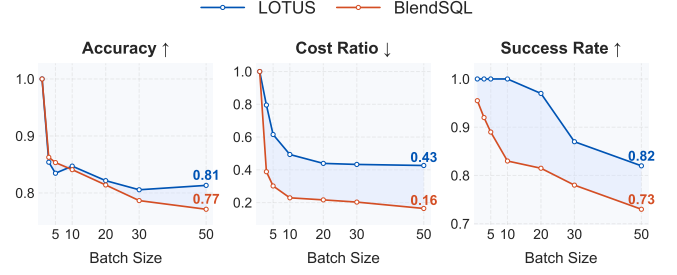
The improvements introduced by AutoHyQ increase with task difficulty: the harder the query, the more the system benefits from structured reasoning guidance. This suggests a promising future direction—*automatically routing queries of varying difficulty levels to subsets of component*—potentially achieving higher overall accuracy while maintaining high execution efficiency.

5.5 Cost Analysis

Solving a hybrid query typically requires hundreds of thousands to millions of tokens. Therefore, in addition to accuracy, execution cost has become a key performance metric, with most of the consumption concentrated on semantic-based data processing. In this section, we analyze costs from two perspectives: (1) whether cost-aware operator sequence optimization can substantially reduce execution overhead during reasoning; and (2) what fundamental cost bottlenecks remain when LLMs are repeatedly invoked for semantic data processing.

As discussed in Section 3.4, we apply operator reordering before answer generation to minimize the input size of semantic operators. To verify its effectiveness, we compare our optimized sequences with execution orders reconstructed by LLMs. Specifically, for each query, we randomly shuffle the optimized operator sequence and provide the unordered set of operators (with fixed parameters) to the LLM, instructing it to produce a valid and efficient ordering. We quantify efficiency using Cost Ratio, defined as the token cost of the LLM-generated plan divided by that of our optimized sequence. Only outputs that exactly follow the given operators, parameters, and overall logic are retained for analysis, ensuring the reliability of cost measurements.

Table 5 shows that LLM-generated plans incur substantially higher costs, reaching 2.59–6.03× that of our optimized reasoning process. Fine-tuning notably improves planning efficiency, with Qwen3-8B-sft demonstrating the strongest cost control capability. Meanwhile, GPT-5.1 achieves excellent performance, benefiting from its inherently strong reasoning capacity. These results indicate that optimizing operator sequencing can significantly reduce cost and that leveraging more powerful models or applying targeted supervised fine-tuning are effective solutions.

**Figure 9: Batch Processing Performance**

It is worth noting that token consumption is overwhelmingly dominated by **prompt tokens**. This imbalance stems from the nature of semantic operators (e.g., *sem_filter*), which rely on long, instruction-heavy prompts but generate extremely short outputs (e.g., true/false). Moreover, operators typically process each row independently and repeatedly use almost identical prompts, leading to substantial redundancy. A natural direction for cost reduction is **batch processing**, where multiple table entries are embedded into a single prompt to reuse instructions and reduce the number of LLM calls. To evaluate its feasibility, we conduct experiments with GPT-4o-mini under different batch sizes, using the non-batched setting as the baseline for cost and accuracy. As the batch size increases, the model may occasionally produce an output sequence with a mismatch between the number of results and the number of input entries. We treat such cases as failures and report the success rate across varying batch sizes.

Figure 9 compares the impact of batch processing on both LOTUS and BlendSQL. For both methods, increasing the batch size substantially reduces execution cost, confirming that batch processing is an effective strategy for amortizing the overhead of semantic operators. Because BlendSQL includes examples in the prompt, it benefits more from batch processing in terms of cost reduction. However, BlendSQL exhibits noticeably worse robustness under batching. As batch size increases, both answer accuracy and execution success rate degrade more rapidly than for LOTUS.

These results confirm that batch processing can substantially reduce costs but also reveal current limitations of LLMs in maintaining semantic fidelity at scale. Addressing this issue will be key to enabling cost-efficient hybrid query solving.

6 Conclusion

This paper introduces HyQBench, a large-scale and diverse benchmark built through automated data generation. We provide AutoHyQ, a pipeline that supports end-to-end automation and fair comparisons of different hybrid query solutions. Extensive experiments reveal the key limitations and challenges that current LLMs face in solving hybrid queries. We hope these findings provide insights and guidance for developing more robust and efficient systems for hybrid queries.

7 Acknowledgments

Chen Lin is the corresponding author. This work was supported by the Natural Science Foundation of China (No.62432011), Fujian Provincial Natural Science Foundation of China(No. 2025J010001)

References

- [1] Asim Biswal, Liana Patel, Siddarth Jha, Amog Kamsetty, Shu Liu, Joseph E. Gonzalez, Carlos Guestrin, and Matei Zaharia. 2025. Text2SQL Is Not Enough: Unifying AI and Databases with TAG. In *CIDR*. www.cidrdb.org.
- [2] Saumya Chaturvedi, Aman Chadha, and Laurent Bindschaedler. 2025. SQL-of-Thought: Multi-agentic Text-to-SQL with Guided Error Correction. arXiv:2509.00581 [cs] doi:10.48550/arXiv.2509.00581
- [3] Si-An Chen, Lesly Miculicich, Julian Martin Eisenschlos, Zifeng Wang, Zilong Wang, Yanfei Chen, Yasuhisa Fujii, Hsuan-Tien Lin, Chen-Yu Lee, and Tomas Pfister. 2025. TableRAG: Million-Token Table Understanding with Language Models. In *Proceedings of the 38th International Conference on Neural Information Processing Systems (NIPS '24, Vol. 37)*. Curran Associates Inc., Red Hook, NY, USA, 74899–74921.
- [4] Wenhui Chen, Ming-Wei Chang, Eva Schlinger, William Wang, and William W Cohen. 2021. Open Question Answering over Tables and Text. *Proceedings of ICLR 2021* (2021).
- [5] Wenhui Chen, Hanwen Zha, Zhiyu Chen, Wenhan Xiong, Hong Wang, and William Yang Wang. 2020. HybridQA: A Dataset of Multi-Hop Question Answering over Tabular and Textual Data. In *Findings of the Association for Computational Linguistics: EMNLP 2020*, Trevor Cohn, Yulan He, and Yang Liu (Eds.). Association for Computational Linguistics, Online, 1026–1036. doi:10.18653/v1/2020.findings-emnlp.91
- [6] Zhoujun Cheng, Tianbao Xie, Peng Shi, Chengzu Li, Rahul Nadkarni, Yushi Hu, Caiming Xiong, Dragomir Radev, Mari Ostendorf, Luke Zettlemoyer, Noah A. Smith, and Tao Yu. 2023. Binding Language Models in Symbolic Languages. *ICLR abs/2210.02875* (2023).
- [7] Colin B. Clement, Matthew Bierbaum, Kevin P. O’Keeffe, and Alexander A. Alemi. 2019. On the Use of ArXiv as a Dataset. arXiv:1905.00075 [cs] doi:10.48550/arXiv.1905.00075
- [8] DeepSeek-AI. 2024. DeepSeek-V3 Technical Report. arXiv:2412.19437 [cs.CL] https://arxiv.org/abs/2412.19437
- [9] Google Gemini Team. 2025. Gemini 2.5: Pushing the Frontier with Advanced Reasoning, Multimodality, Long Context, and Next Generation Agentic Capabilities. arXiv:2507.06261 [cs] doi:10.48550/arXiv.2507.06261
- [10] Parker Glenn, Parag Dake, Liang Wang, and Preethi Raghavan. 2024. BlendSQL: A Scalable Dialect for Unifying Hybrid Question Answering in Relational Algebra. In *Findings of the Association for Computational Linguistics: ACL 2024*, Lun-Wei Ku, Andre Martins, and Vivek Srikumar (Eds.). Association for Computational Linguistics, Bangkok, Thailand, 453–466. doi:10.18653/v1/2024.findings-acl.25
- [11] Jonathan Herzig, Pawel Krzysztow Nowak, Thomas Müller, Francesco Piccinno, and Julian Eisenschlos. 2020. TaPas: Weakly Supervised Table Parsing via Pre-training. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, Dan Jurafsky, Joyce Chai, Natalie Schluter, and Joel Tetreault (Eds.). Association for Computational Linguistics, Online, 4320–4333. doi:10.18653/v1/2020.acl-main.398
- [12] Zijin Hong, Zheng Yuan, Qinggang Zhang, Hao Chen, Junnan Dong, Feiran Huang, and Xiao Huang. 2025. Next-Generation Database Interfaces: A Survey of LLM-based Text-to-SQL. arXiv:2406.08426 [cs] doi:10.48550/arXiv.2406.08426
- [13] Edward J Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. 2022. LoRA: Low-Rank Adaptation of Large Language Models. In *International Conference on Learning Representations*. https://openreview.net/forum?id=nZKeVfYf9
- [14] Mohit Iyyer, Wen-tau Yih, and Ming-Wei Chang. 2017. Search-Based Neural Structured Learning for Sequential Question Answering. In *Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, Regina Barzilay and Min-Yen Kan (Eds.). Association for Computational Linguistics, Vancouver, Canada, 1821–1831. doi:10.18653/v1/P17-1167
- [15] Nengzheng Jin, Joanna Siebert, Dongfang Li, and Qingcai Chen. 2022. A Survey on Table Question Answering: Recent Advances. arXiv:2207.05270 [cs] doi:10.48550/arXiv.2207.05270
- [16] Ravi Kashyap. 2023. Machine Learning in Google Cloud Big Query Using SQL. *International Journal of Computer Science and Engineering* 10, 5 (May 2023), 17–25. doi:10.14445/23488387/IJCSE-V10I5P103
- [17] Udesch Kumarasinghe, Tyler Liu, Chunwei Liu, and Walid G. Aref. 2026. iPDB – Optimizing SQL Queries with ML and LLM Predicates. arXiv:2601.16432 [cs] doi:10.48550/arXiv.2601.16432
- [18] Fangyu Lei, Jixuan Chen, Yuxiao Ye, Ruisheng Cao, Dongchan Shin, Hongjin Su, Zhaoqing Suo, Hongcheng Gao, Wenjing Hu, Pengcheng Yin, Victor Zhong, Caiming Xiong, Ruoxi Sun, Qian Liu, Sida Wang, and Tao Yu. 2024. Spider 2.0: Evaluating Language Models on Real-World Enterprise Text-to-SQL Workflows. In *The Thirteenth International Conference on Learning Representations*.
- [19] Boyan Li, Yuyu Luo, Chengliang Chai, Guoliang Li, and Nan Tang. 2024. The Dawn of Natural Language to SQL: Are We Fully Ready? *Proceedings of the VLDB Endowment* 17, 11 (July 2024), 3318–3331. doi:10.14778/3681954.3682003
- [20] Jinyang Li, Binyuan Hui, Ge Qu, Jiaxi Yang, Binhua Li, Bowen Li, Bailin Wang, Bowen Qin, Ruiying Geng, Nan Huo, et al. 2024. Can llm already serve as a database interface? a big bench for large-scale database grounded text-to-sqls. *Advances in Neural Information Processing Systems* 36 (2024).
- [21] Peng Li, Yeye He, Dror Yashar, Weiwei Cui, Song Ge, Haidong Zhang, Danielle Rifinski Fainman, Dongmei Zhang, and Surajit Chaudhuri. 2024. Table-GPT: Table Fine-tuned GPT for Diverse Table Tasks. *Proceedings of the ACM on Management of Data* 2, 3 (May 2024), 1–28. doi:10.1145/3654979
- [22] Weizhe Lin, Rexhina Blloshmi, Bill Byrne, Adria de Gispert, and Gonzalo Iglesias. 2023. An Inner Table Retriever for Robust Table Question Answering. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, Anna Rogers, Jordan Boyd-Graber, and Naoaki Okazaki (Eds.). Association for Computational Linguistics, Toronto, Canada, 9909–9926. doi:10.18653/v1/2023.acl-long.551
- [23] Xinyu Liu, Shuyu Shen, Boyan Li, Peixian Ma, Runzhi Jiang, Yuxin Zhang, Ju Fan, Guoliang Li, Nan Tang, and Yuyu Luo. 2024. A Survey of NL2SQL with Large Language Models: Where are we, and where are we going? *arXiv preprint arXiv:2408.05109* (2024).
- [24] Ruilin Luo, Liyuan Wang, Binghui Lin, Zicheng Lin, and Yujiu Yang. 2024. PTD-SQL: Partitioning and Targeted Drilling with LLMs in Text-to-SQL. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, Yaser Al-Onaizan, Mohit Bansal, and Yun-Nung Chen (Eds.). Association for Computational Linguistics, Miami, Florida, USA, 3767–3799. doi:10.18653/v1/2024.emnlp-main.221
- [25] Rishabh Misra. 2022. News Category Dataset. *arXiv preprint arXiv:2209.11429* (2022).
- [26] Rishabh Misra and Jigyasa Grover. 2021. *Sculpting Data for ML: The first act of Machine Learning*.
- [27] Jianmo Ni, Jiacheng Li, and Julian McAuley. 2019. Justifying Recommendations Using Distantly-Labeled Reviews and Fine-Grained Aspects. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, Kentaro Inui, Jing Jiang, Vincent Ng, and Xiaojun Wan (Eds.). Association for Computational Linguistics, Hong Kong, China, 188–197. doi:10.18653/v1/D19-1018
- [28] OpenAI. 2024. GPT-4 Technical Report. arXiv:2303.08774 [cs] doi:10.48550/arXiv.2303.08774
- [29] OpenAI. 2025. GPT-5.1: A smarter, more conversational ChatGPT. https://openai.com/index/gpt-5-1
- [30] Panupong Pasupat and Percy Liang. 2015. Compositional Semantic Parsing on Semi-Structured Tables. In *Proceedings of the 53rd Annual Meeting of the Association for Computational Linguistics and the 7th International Joint Conference on Natural Language Processing (Volume 1: Long Papers)*, Chengqing Zong and Michael Strube (Eds.). Association for Computational Linguistics, Beijing, China, 1470–1480. doi:10.3115/v1/P15-1142
- [31] Liana Patel, Siddharth Jha, Melissa Pan, Harshit Gupta, Parth Asawa, Carlos Guestrin, and Matei Zaharia. 2025. Semantic Operators and Their Optimization: Enabling LLM-Based Data Processing with Accuracy Guarantees in LOTUS. *Proceedings of the VLDB Endowment* 18, 11 (July 2025), 4171–4184. doi:10.14778/3749646.3749685
- [32] Mohammadreza Pourreza, Hailong Li, Ruoxi Sun, Yeounoh Chung, Shayan Talaei, Gaurav Tarlok Kakkar, Yu Gan, Amin Saberi, Fatma Ozcan, and Sercan O. Arik. 2024. CHASE-SQL: Multi-Path Reasoning and Preference Optimized Candidate Selection in Text-to-SQL. In *The Thirteenth International Conference on Learning Representations*.
- [33] Mohammadreza Pourreza and Davood Rafiei. 2023. DIN-SQL: Decomposed in-Context Learning of Text-to-SQL with Self-Correction. In *Proceedings of the 37th International Conference on Neural Information Processing Systems (NIPS '23)*. Curran Associates Inc., Red Hook, NY, USA, 36339–36348.
- [34] Mohammadreza Pourreza and Davood Rafiei. 2024. DTS-SQL: Decomposed Text-to-SQL with Small Large Language Models. In *Findings of the Association for Computational Linguistics: EMNLP 2024*, Yaser Al-Onaizan, Mohit Bansal, and Yun-Nung Chen (Eds.). Association for Computational Linguistics, Miami, Florida, USA, 8212–8220. doi:10.18653/v1/2024.findings-emnlp.481
- [35] Mark Raasveldt and Hannes Mühleisen. 2019. DuckDB: An Embeddable Analytical Database. In *Proceedings of the 2019 International Conference on Management of Data (SIGMOD '19)*. Association for Computing Machinery, New York, NY, USA, 1981–1984. doi:10.1145/3299869.3302012
- [36] Jie Shi, Bo Xu, Jiaqing Liang, Yanghua Xiao, Jia Chen, Chenhao Xie, Peng Wang, and Wei Wang. 2025. Gen-SQL: Efficient Text-to-SQL By Bridging Natural Language Question And Database Schema With Pseudo-Schema. In *Proceedings of the 31st International Conference on Computational Linguistics*, Owen Rambow, Leo Wanner, Marianna Apidianaki, Hend Al-Khalifa, Barbara Di Eugenio, and Steven Schockaert (Eds.). Association for Computational Linguistics, Abu Dhabi, UAE, 3794–3807.
- [37] Vladislav Shkapenyuk, Divesh Srivastava, Theodore Johnson, and Parisa Ghane. 2025. Automatic Metadata Extraction for Text-to-SQL. arXiv:2505.19988 [cs] doi:10.48550/arXiv.2505.19988
- [38] Qwen Team. 2025. Qwen2.5 Technical Report. arXiv:2412.15115 [cs.CL] https://arxiv.org/abs/2412.15115
- [39] Qwen Team. 2025. Qwen3 Technical Report. arXiv:2505.09388 [cs.CL] https://arxiv.org/abs/2505.09388

- [40] Pengfei Wang, Baolin Sun, Xuemei Dong, Yaxun Dai, Hongwei Yuan, Mengdie Chu, Yingqi Gao, Xiang Qi, Peng Zhang, and Ying Yan. 2025. Agentar-Scale-SQL: Advancing Text-to-SQL through Orchestrated Test-Time Scaling. arXiv:2509.24403 [cs] doi:10.48550/arXiv.2509.24403
- [41] Tomer Wolfson, Mor Geva, Ankit Gupta, Matt Gardner, Yoav Goldberg, Daniel Deutch, and Jonathan Berant. 2020. Break It Down: A Question Understanding Benchmark. *Transactions of the Association for Computational Linguistics* 8 (2020), 183–198. doi:10.1162/tacl_a_00309
- [42] Fangzhao Wu, Ying Qiao, Jiun-Hung Chen, Chuhan Wu, Tao Qi, Jianxun Lian, Danyang Liu, Xing Xie, Jianfeng Gao, Winnie Wu, and Ming Zhou. 2020. MIND: A Large-scale Dataset for News Recommendation. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*. Association for Computational Linguistics, Online. doi:10.18653/v1/2020.acl-main.331
- [43] Tao Yu, Rui Zhang, Kai Yang, Michihiro Yasunaga, Dongxu Wang, Zifan Li, James Ma, Irene Li, Qingning Yao, Shanelle Roman, Zilin Zhang, and Dragomir Radev. 2018. Spider: A Large-Scale Human-Labeled Dataset for Complex and Cross-Domain Semantic Parsing and Text-to-SQL Task. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*. Ellen Riloff, David Chiang, Julia Hockenmaier, and Jun'ichi Tsujii (Eds.). Association for Computational Linguistics, Brussels, Belgium, 3911–3921. doi:10.18653/v1/D18-1425
- [44] Hongwei Yuan, Xiu Tang, Ke Chen, Lidan Shou, Gang Chen, and Huan Li. 2025. CogSQL: A Cognitive Framework for Enhancing Large Language Models in Text-to-SQL Translation. *Proceedings of the AAAI Conference on Artificial Intelligence* 39, 24 (April 2025), 25778–25786. doi:10.1609/aaai.v39i24.34770
- [45] Fuheng Zhao, Divyakant Agrawal, and Amr El Abbadi. 2025. Hybrid Querying Over Relational Databases and Large Language Models. In *CIDR*. www.cidrdb.org.
- [46] Victor Zhong, Caiming Xiong, and Richard Socher. 2017. Seq2SQL: Generating Structured Queries from Natural Language using Reinforcement Learning. *CoRR* abs/1709.00103 (2017).

A Information of Source Tables

The content and approximate attributes of the source tables are as follows:

BIRD: This is a large-scale benchmark for the text-to-SQL task, containing 12,751 text-to-SQL pairs and 95 databases, with a total size of 33.4 GB and covering 37 domains. The BIRD database is highly similar to real-world datasets, with multiple tables and complex foreign-key relationships, reflecting the complexity of working in real-world multi-table environments. We select databases from the Mini-Dev dataset of BIRD, a high-quality subset with complete table relationships and reasonable complexity, thereby providing a solid foundation for our dataset.

Table 6: Source table statistics

Database	#Table	Avg. #Rows	Avg. #Columns
california_schools	3	1312.33	29.67
card_games	6	1241.00	19.17
codebase_community	8	1693.75	8.88
debit_card_specializing	5	918.20	4.20
european_football_2	7	666.43	28.43
financial	8	1019.63	6.88
formula_1	13	713.54	7.23
student_club	8	204.25	6.00
superhero	10	305.10	3.10
thrombosis_prediction	3	702.00	21.33
toxicology	4	835.75	2.75
arxiv	1	1000.00	13.00
microsoft_news	1	1000.00	6.00
news_category	1	1000.00	6.00
amazon_review	1	1000.00	11.00

Arxiv Dataset: arXiv provides open access to academic articles for the public and the research community, serving a broad range of

fields. The dataset is a repository of 1.7 million articles with related attributes such as authors, categories, abstracts, and full-text PDFs. Because the whole dataset is very large, we used the metadata file, which includes entries for each paper containing the title, authors, abstract, and other details.

MIND: This is a large-scale dataset for news recommendation research. The data is collected from anonymous behavioral logs on the Microsoft News website. MIND contains around 160,000 English news articles and over 15 million interaction logs generated by 1 million users. Each news article includes rich text content, including titles, abstracts, body content, categories, and entities.

News Category Dataset: This dataset contains approximately 210,000 HuffPost news headlines from 2012 to 2022. It is one of the largest news datasets and can serve as a benchmark for various computational linguistics tasks. Around 200,000 news headlines are from 2012 to May 2018, and 10,000 headlines are from May 2018 to 2022. Each data point includes various attributes of a news article, including category, headline, short description, etc.

Amazon Review Data: This dataset is an updated version of the Amazon review dataset released in 2014. Like previous versions, it includes reviews (ratings, text, helpfulness votes) and product metadata (descriptions, category information, price, brand, and image features). We selected a 5-core table from the dataset, Amazon Fashion, and organized it to extract important and informative attributes, such as reviewer ID, reviewer name, review time, product data, and review content.

Algorithm 1: Predicate Pushdown

Input: A filter node n and its child node c in the query tree
Output: Modified tree structure with n pushed down if applicable

```

1 Function PushDownFilter( $n, c$ ):
2   if not IsCommutative( $c$ ) then
3     return;
4   end
5   if  $c.op \notin \{join, sem\_join\}$  then
6     SwapNodes( $n, c, c.child$ );
7   else if  $n.op = filter$  then
8      $t \leftarrow FindSubtree(n, c)$ ;
9     if  $t \neq None$  then
10      SwapNodes( $n, c, t$ );
11    end
12  end
13  else if  $n.op = sem\_filter$  then
14     $t \leftarrow FindSubtree(n, c)$ ;
15    if HasCommutativeSemJoin( $t$ ) then
16      SwapNodes( $n, c, t$ );
17    end
18  end

```

B Algorithm Details

We use *Filter*, *Join*, etc., to denote operator types, and *filter*, *sem_filter*, etc., to denote specific operators.

Algorithm 2: Reorder

Input: Query tree $\mathcal{T}$
Output: Query tree with reordered operators

```

1 Function ReorderSameTypeOperators( $\mathcal{T}$ ):
2    $C \leftarrow \text{FindSameTypeOperators}(\mathcal{T});$ 
3   foreach  $chain \in C$  do
4     for  $i \leftarrow 0$  to  $|chain| - 2$  do
5       for  $j \leftarrow i$  to  $|chain| - 2$  do
6         if  $\text{EstimateCost}(chain[j]) <$ 
            $\text{EstimateCost}(chain[j + 1])$  then
7            $\text{SwapNodes}(chain[j], chain[j + 1],$ 
              $chain[j + 1].\text{children});$ 
8           end
9         end
10      end
11    end
12 Function EstimateCost( $n$ ):
13   if  $n.op = \text{sem\_filter}$  then
14      $t \leftarrow \text{FindBaseTable}(n);$ 
15      $c \leftarrow \text{target column of } n;$ 
16     return  $\text{NumDistinct}(t[c])$ 
17   else if  $n.op = \text{sem\_join}$  then
18      $t_1, t_2 \leftarrow \text{input FindBaseTable}(n);$ 
19      $c_1, c_2 \leftarrow \text{join columns of } n;$ 
20     return  $\text{NumDistinct}(t_1[c_1]) \times$ 
        $\text{NumDistinct}(t_2[c_2])$ 
21   else
22     return 0;
23   end

```

We focus on optimizing **Filter** and **Join** operators, which have the most significant impact on the number of rows processed in hybrid queries. These operators appear in nearly every query and constitute a substantial portion of the overall reasoning process. In our dataset, their proportion among all operators reaches 53.8%. Inspired by classical optimization techniques such as predicate pushdown and adjacent-operator reordering, we adapt these strategies to the hybrid query setting by explicitly accounting for the distinct behavior of semantic operators.

B.1 Predicate Pushdown

To improve execution efficiency, we first perform predicate pushdown for **Filter** operators. The detailed procedure is shown in Algorithm 1. We define commutative operators as operators whose

execution order can be safely swapped with a **Filter** operator without affecting the semantic correctness of the final result. Typical examples include operators such as **Join**, **Map**, and **Group**. In particular, for the **Join** operator, the filter must determine the pushdown position based on the contents of its child nodes.

Given a **Filter** node n and its child node c , Algorithm 1 determines whether n can be pushed below c based on their operator types:

- (1) Child node is a commutative non-join operator
 If c belongs to commutative operators but is a non-join operator—e.g., **Map** or **Group**—then the two operators can be directly swapped. This allows the **Filter** operator to execute earlier, thereby reducing downstream data volume.
- (2) Child node is a *join* operator, and n is a *filter*
 When c is a *join*, pushing down a *filter* requires locating the specific subtree that the filtering condition applies to. Algorithm 1 uses the FindSubtree procedure to identify the correct branch and then pushes the *filter* operator only into that branch, preserving semantic correctness.
- (3) Child node is a *join*, and n is a *sem_filter*
sem_filter has a higher computational cost and a more restricted applicability. When encountering a join operator, we check whether the subtree contains at least one commutative *sem_join* (using HasCommutativeSemJoin). Only in that case is it safe and beneficial to push the *sem_filter* down; otherwise, the original order is preserved to avoid introducing unnecessary LLM calls.

B.2 Reordering Adjacent Same-Type Operators

After pushing down the Filter operators, we further optimize the execution order of adjacent **Filter** and **Join** operators. Since our reasoning pipeline incorporates a deduplication mechanism, the actual volume of data processed by each operator is closely tied to the number of distinct values in the relevant columns. To leverage this, we first ensure that traditional filter and join operators are executed prior to their semantic counterparts, namely *sem_filter* and *sem_join*. This prioritization aims to reduce the amount of data processed by more expensive semantic operators. For each *sem_filter* and *sem_join*, we identify the corresponding base table and the columns on which they operate. For *sem_filter*, we compute the number of distinct values in the target column after deduplication and use this as a cost estimate. For *sem_join*, we compute the product of the distinct-value counts in the join columns of the two input tables as the cost estimate. Finally, we sort all *sem_filter* and *sem_join* operators by their estimated costs in ascending order and reorder them in the execution sequence accordingly.